10X Front-end სადიპლომო პროექტი - „10X Shop“

Product Requirements Document (PRD)

ინდივიდუალური სადიპლომო პროექტი

ვერსია 1.1 · 2026 წლის ნაკადი

სარჩევი

0. შესავალი **2**

პროექტის განხორციელების წესები **2**

1. AI-ის გამოყენება ნებადართულია სრულად. 3

2. პროექტის შესრულების რეკომენდებული ეტაპები **3**

3. რა უნდა გააკეთო: 28 ნაბიჯი თანმიმდევრობით **4**

4. ჩაბარება და ვადები **6**

0. შესავალი

რას ქმნი: „10X Shop“ ონლაინ მაღაზიის SPA React + TypeScript + Tailwind CSS v4-ზე, DummyJSON API-ით.

CORE: ლოგინი, კატალოგი, პროდუქტის დეტალები, კალათა (რაოდენობის შეცვლა და წაშლა), checkout, responsive ინტერფეისი.

FULL: ძებნა/ფილტრაცია/სორტი, Orders, Profile და დამატებითი ფუნქციები.

რას აბარებ: საჯარო GitHub რეპო 10x-shop-{სახელი}, ლაივ ბმული (Vercel / Netlify), კოდი, README.md (EN), მოკლე PLAN.md და REPORT.md, deploy და შინაარსიანი Git ისტორია.

როგორ ფასდება: ხუთი მოდული ცალკე მოწმდება: React, TypeScript, Tailwind CSS, ოპტიმიზაცია და პრაქტიკული პროექტი. ჩათვლის მინიმუმი არის CORE ჩეკლისტი. FULL/BONUS მაღალი ქულისთვისაა.

მოდული	CORE ერთ ხაზად (ჩათვლის მინიმუმი)	FULL (4-5 ქულის ზონა)
React	Components/Props/children, useState/useEffect, map/key, controlled Login/Checkout, Router, Context და მარტივი Cart.	დამატებითი custom hooks, რთული reducer, მოწინავე UX/მარშრუტები. Formik + Yup (Login ფორმის გაფართოებული მართვა და ვალიდაცია)
TypeScript	Props/state/API ტიპები, strict TypeScript, typecheck = 0	generics, ApiClient, OOP კლასები, type guards, unions
ოპტიმიზაცია	გვერდები responsive, Error Boundary, DEBUG-LOG, შეცდომების საბაზისო გამართვა	lazy routes/images
პრაქტიკული პროექტი	README, PLAN.md, deploy და Git-ის რეალური ისტორია	
Tailwind CSS	სავალდებულო v4 ინტეგრაცია, utility styling, grid/flex, responsive, hover/focus	@theme, dark mode, advanced variants, custom utilities

ვადები: პროექტის 3 კვირა — 17 ოქტომბერი–06 ნოემბერი, 2026; თეორიული ტესტირება — 05 ნოემბერს; GitHub რეპოზიტორიის გაგზავნის ბოლო ვადა — 06 ნოემბერი, 23:59; დაცვა — 07–08 ნოემბერს. პრიორიტეტი: ჯერ CORE, შემდეგ FULL.

პროექტის განხორციელების წესები

პროექტი იყოფა CORE, FULL და BONUS ფუნქციონალად.

CORE არის სავალდებულო მინიმალური ფუნქციონალი. **FULL** წარმოადგენს გაფართოებულ მოთხოვნებს. **BONUS** არასავალდებულო დამატებითი შესაძლებლობებია.

პროექტის განხორციელებისას დაცული უნდა იყოს შემდეგი წესები:

1. პირველ ეტაპზე განხორციელდეს მხოლოდ CORE ფუნქციონალი.
2. FULL და BONUS ფუნქციები არ განხორციელდეს დამატებითი მითითების გარეშე.

3. თუ დეტალურ ტექნიკურ მაგალითსა და სექციის CORE/FULL განსაზღვრებას შორის წინააღმდეგობაა, უპირატესობა ენიჭება CORE/FULL მოთხოვნების განმარტებას.
4. TypeScript-ის ტიპები უნდა შეესაბამებოდეს რეალურად განხორციელებულ მონაცემებსა და ფუნქციებს.
5. Tailwind CSS-ის სტილიზაცია უნდა განხორციელდეს რეალურად არსებული კომპონენტებისთვის.
6. Login წარმოადგენს ლოკალურ React Context-ზე დაფუძნებულ სიმულაციას. რეალური ავტორიზაცია და გარე authentication API არ გამოიყენება.
7. Checkout წარმოადგენს controlled form-ს და შეკვეთის ლოკალურ სიმულაციას. რეალური გადახდის სისტემა არ გამოიყენება.
8. არ უნდა დაემატოს დოკუმენტით გაუთვალისწინებელი ბიბლიოთეკები და ფუნქციები, თუ ისინი CORE მოთხოვნების შესრულებისთვის აუცილებელი არ არის.
9. ყველა ფუნქცია უნდა შეესაბამებოდეს შესაბამის Acceptance Criteria-ს.
10. თუ მოთხოვნა ბუნდოვანია ან წინააღმდეგობრივია, განხორციელებამდე საჭიროა მისი დაზუსტება.

1. AI-ის გამოყენება ნებადართულია. სრულად.

შეგიძლია გამოიყენო ნებისმიერი AI ინსტრუმენტი (Claude, ChatGPT, Copilot, Gemini...) ნებისმიერ ეტაპზე.ეს თანამედროვე დეველოპერის სამუშაო პროცესია. სამი რამ კი ნამდვილად შენი პასუხისმგებლობაა:

- **1. პროდუქტი უნდა მუშაობდეს.** AI-ის დაწერილი გაუმართავი კოდი შენი გაუმართავი კოდია. პროდუქტი თუ არ მუშაობს, არ აქვს მნიშვნელობა, რამდენი დრო დახარჯე.
- **2. უნდა შეგეძლოს ახსნა, რა გააკეთე.** დაცვაზე შემფასებელი შენს კოდზე დაგისვამს კითხვებს. თუ ვერ ხსნი შენივე კოდს – ქულა იკლებს, რამდენად კარგადაც არ უნდა მუშაობდეს აპლიკაცია.
- **3. პროცესი შენი უნდა იყოს.** გეგმა, commit-ები, ყოველკვირეული ანგარიშები – ეს შენი პროფესიული ქცევის მტკიცებულებაა. AI დაგეხმარება ფორმულირებაში, მაგრამ რა, როდის და რატომ გააკეთე – ამ გადაწყვეტილებებს შენ იღებ და შენ ასაბუთებ.

2. პროექტის შესრულების რეკომენდებული ეტაპები

პროექტის შესასრულებლად განკუთვნილია 3 კვირა.

კვირა 1 (17–23 ოქტომბერი): პროექტის კონფიგურაცია, React + TypeScript + Tailwind CSS, ძირითადი კომპონენტები, Login, Routing და Catalog.
კვირა 2 (24–30 ოქტომბერი): Product Details, Cart, Checkout, React State Management და მონაცემების ტიპიზაცია.
კვირა 3 (31 ოქტომბერი – 6 ნოემბერი): Responsive დიზაინის დასრულება, ძირითადი ფუნქციონალის შემოწმება, დოკუმენტაცია, Deploy და სურვილისამებრ FULL ფუნქციები.
ეტაპები რეკომენდებულია და არ წარმოადგენს სავალდებულო ტექნიკურ თანმიმდევრობას.

პრიორიტეტი ენიჭება CORE მოთხოვნების სრულად შესრულებას. პროექტის ტექნიკური

დეტალები განსაზღვრულია დანართში A.

3. რა უნდა გააკეთო: 28 ნაბიჯი თანმიმდევრობით

CORE: მინიმალური ჩათვლისთვის სავალდებულო ფუნქციონალია;
FULL: დამატებითი შესაძლებლობები მაღალი შეფასებისთვის.

კვირა 1 — ფუნდამენტი, Login და Catalog

№	ნაბიჯი / ამოცანა	შემოწმებადი შედეგი	დონე
1	Vite + React + TypeScript პროექტის შექმნა; strict tsconfig, ძირითადი scripts, Tailwind CSS v4.	npm run typecheck და npm run build შეცდომების გარეშე სრულდება; Tailwind utility classes მუშაობს.	CORE
2	მოკლე PLAN.md, GitHub რეპოზიტორია და ამოცანების დაყოფა.	PLAN.md აღწერს CORE მიზნებს და 3 კვირის ეტაპებს; Git ისტორია შინაარსიანია.	CORE
3	საჭირო TypeScript ტიპები: Product, ProductsResponse, CartItem/CartState, მარტივი Login და Checkout ფორმის ველები.	Props, state და API მონაცემები ტიპიზებულია; ზედმეტი FULL ტიპების შექმნა არ მოითხოვება.	CORE
4	ფუნქციური კომპონენტები, Layout/Header, Props/children და ძირითადი React Router.	მუშაობს /, /login, /products/:id, /cart, /checkout და *; ნავიგაცია reload-ის გარეშეა.	CORE
5	AuthContext მარტივი login/logout state-ით და Login controlled form-ით.	Username/Password required ვალიდაცია მუშაობს; valid submit ცვლის Context state-ს;	CORE
6	Checkout-ის სტუმრისთვის მარტივი redirect.	სტუმარი /checkout-დან მიდის /login-ზე, შესვლის შემდეგ ბრუნდება Checkout-ზე; გამოიყენება React state/Router.	CORE
7	Catalog: GET /products, useEffect, loading/error/empty მდგომარეობები.	API-დან პროდუქტები ჩანს; state-ის ცვლილებები და შეცდომის ტექსტები სწორად გამოისახება.	CORE
8	ProductCard და ProductGrid: props, map(), სტაბილური key და Tailwind responsive grid.	პროდუქტები კომპონენტებად არის დარენდერებული; ბარათიდან დეტალებზე გადასვლა მუშაობს.	CORE
9	Search, category, sort, pagination და URL state (სურვილისამებრ).	მხოლოდ განხორციელების შემთხვევაში მუშაობს შესაბამისი API და URL ქცევა; CORE-ს არ აფერხებს.	FULL

კვირა 2 — Product, Cart და Checkout

№	ნაბიჯი / ამოცანა	შემოწმებადი შედეგი	დონე
10	Product Details: useParams + GET /products/:id.	პროდუქტის სათაური, სურათი, აღწერა, ფასი და მარაგი ჩანს; არასწორ ID-ზე აპლიკაცია არ იშლება.	CORE
11	Product-ის ინტერაქტიული ელემენტები: მარტივი ფოტო/რაოდენობის არჩევა და Add to Cart.	useState და event handlers მუშაობს; პროდუქტი ემატება საერთო კალათას.	CORE
12	CartContext: საერთო items state, add/remove/setQuantity და immutable updates.	Cart-ის ცვლილებები Header badge-სა და Cart გვერდზე ერთნაირად ჩანს;	CORE
13	Cart გვერდი: პროდუქტების სია, quantity, remove და subtotal/total.	რაოდენობის ცვლილება და წაშლა განაახლებს ჯამს; ცარიელი კალათა გასაგებად ჩანს.	CORE
14	Checkout: Full name, Email, Address, City controlled fields; მარტივი validation.	onChange/useState/onSubmit/preventDefault მუშაობს; ცარიელი required ველები submit-ს ბლოკავს.	CORE
15	შეკვეთის ლოკალური დადასტურება.	Place order აჩვენებს Thank you/summary-ს და ცლის კალათას;	CORE
16	React-ის განვლილი კონცეფციების შემოწმება.	ჩანს props, children, callback props, map/key, conditional rendering, Context, useEffect dependencies და immutable state.	CORE
17	Reviews, Related Products, Promo Code და რთული Cart logic.	არასავალდებულო ფუნქციები სწორად მუშაობს, თუ სტუდენტი მათ აირჩევს.	FULL
18	Orders/Profile, დამატებითი custom hooks და Login ფორმის გაფართოება Formik + Yup-ის გამოყენებით.	Formik-ით შექმნილი Login ფორმა იყენებს Yup Validation Schema-ს, ამუშავებს ველების ცვლილებებსა და შეცდომებს.	FULL

კვირა 3 — დასრულება, ხარისხი და ჩაბარება (31 ოქტომბერი – 6 ნოემბერი)

№	ნაბიჯი / ამოცანა	შემოწმებადი შედეგი (DoD)	დონე
19	CORE ინტერფეისის Tailwind responsive დასრულება.	Catalog, Product, Cart, Login და Checkout გამართულად ჩანს 375px-სა და desktop-ზე; არაა ჰორიზონტალური scroll.	CORE
20	UI interactive states: hover, focus და საჭიროებისას disabled.	სავალდებულო Tailwind variants რეალურ ღილაკებსა და ფორმის ველებში გამოიყენება.	CORE
21	ძირითადი სამუშაო პროცესის ტესტირება.	Catalog → Product → Cart → Login (საჭიროებისას) → Checkout → confirmation მუშაობს.	CORE

22	Loading/Error/Empty და საბაზისო debugging.	API შეცდომა არ იწვევს აპლიკაციის ჩავარდნას; სტუდენტი ხსნის როგორ იპოვა/გაასწორა პრობლემა.	CORE
23	TypeScript-ის შემოწმება და production build.	strict typecheck = 0 შეცდომა; npm run build წარმატებულია.	CORE
24	Deploy (Vercel/Netlify) და SPA route refresh შემოწმება.	ლაივ ბმული მუშაობს; /products/5 პირდაპირ იხსნება/refresh-დება; ბმული README-შია.	CORE
25	README.md (EN), მოკლე PLAN.md, REPORT.md და რეალური Git ისტორია.	აღწერილია დაყენება/გაშვება, CORE ფუნქციები, გამოყენებული ტექნოლოგიები, შედეგები და შეზღუდვები.	CORE
26	გაფართოებული ოპტიმიზაცია: lazy loading, Lighthouse before/after, cross-browser ანგარიში.	თუ შესრულებულია, დამატებითი შედეგები დოკუმენტირებულია; არ არის React CORE-ის წინაპირობა.	FULL
27	FULL დონის Tailwind/TypeScript შესაძლებლობები: @theme, generics, OOP და სხვ.	სურვილისამებრ შესრულებული მოთხოვნები შეესაბამება შესაბამის მოდულურ სპეციფიკაციას.	FULL
28	საბოლოო QA და რეპოზიტორიის გაგზავნა.	შესრულებულია CORE ჩეკლისტი; საბოლოო კოდი GitHub-ზეა და რეპოზიტორია გაგზავნილია 6 ნოემბერს 23:59-მდე.	CORE

4. ჩაბარება და ვადები

- საჯარო GitHub რეპოზიტორია: 10x-shop-{სახელი}. კოდი, README.md (EN), მოკლე PLAN.md და deploy-ის ბმული.
- npm install && npm run typecheck && npm run build წარმატებით სრულდება;
- ლაივ ბმული (Vercel / Netlify) README-შია და SPA მუშაობს;
- Git ისტორია უნდა ასახავდეს რეალურ, შინაარსიან მუშაობას.
- **პროექტის GitHub რეპოზიტორია:** შესაბამის ფორმაში უნდა გამოიგზავნოს 2026 წლის ***06 ნოემბრის 23:59-მდე*** (საქართველოს დროით). ეს არის კოდის საბოლოო ჩაბარების ვადა.
- **თეორიული ტესტირება:** ჩატარდება ***2026 წლის 05 ნოემბერს*** — 30 კითხვა, 45 წუთი; საკითხები: React, TypeScript, Tailwind CSS, ოპტიმიზაცია. მინიმალური ზღვარი: 60%; ქვიზს გუგლის ფორმით მიიღებ.
- **პროექტის დაცვა:** ჩატარდება **2026 წლის 07–08 ნოემბერს,** შაბათსა და კვირას — ჯგუფების განრიგის მიხედვით. 06 ნოემბრის 23:59-ის შემდეგ შეტანილი ცვლილებები შეფასებაში არ მონაწილეობს.